

ARCHITECT 심사 준비 특강

Architect 양성사무국 김우성



Architect 양성과정 Self-Checklist 리뷰

1. 문제정의
 2. 요구사항
 3. 설계
 4. 구현 / 5. 검증
- 기타. 면접 준비 Tip

1. 문제 정의

• 과제 필요성

- 과제 선정 배경, 과제 목표가 기술되었는지 확인한다.
- Architecting이 필요한 과제인지 확인한다.

• 과제 범위

- 과제 개발 범위, 내용, 기간이 기술되었는지 확인한다.

[Tip]

- 과제 규모는 너무 작지 않게
그러나 과장할 필요는 없음
- 어려운 용어는 짧게 풀이 추가
- 상품기획, FRD 등 과제 관련 문서
상품기획 담당자 리뷰도 도움이 됨
- 과제 목표가 품질속성 반영여부 확인
- 과제에 대해 유추 가능한 제목 선정

[Do]

- 과제의 **중요성**, 필요성
- Architect가 필요한 과제
(난이도/규모)
- Domain에 대한 쉬운 설명
- 배경→필요성→목표 흐름이
공감할 수 있는 구성 (Story Telling)
- 명확한 수치 기반 비즈니스 가치 표현

[Don't]

- 기술 자체에 대한 원론적인 설명
- 장황한 도메인 설명
(핵심만 남기고 부연은 Appendix로)

2. 요구사항

- **요구사항 수집**

- 과제와 연관된 주요 Stakeholder가 모두 정의되었는지 확인한다.
- Stakeholder와 과제 특성에 따른 요구사항 수집 방법이 적절하게 선정되었는지 확인한다.
- 요구사항 수집 계획에 따라 요구사항 수집이 진행되었는지 확인한다.

[Tip]

- 여러 Stakeholder 대상 VoC 수집 결과와 과제 목표의 연관 관계를 명시
- QAW 템플릿을 활용하되 나만의 차별성이 있다면 언급해주면 좋음

2. 요구사항

• 요구사항 도출 (1/2)

- 수집된 요구사항이 정제된 이후에도 기술된 내용이 모호한 것은 없는지 확인한다.
- 요구사항이 과제 특성을 고려하여 의미 있게 분류되었는지 확인한다. (Biz/기술 or 내부/외부 등)
- 요구 사항 분석을 통해 기능 요구사항은 모두 기술되었는지 확인한다.
- 요구 사항 분석을 통해 비 기능 요구 사항(품질 속성)은 모두 기술되었는지 확인한다.
- 제약 사항(존재할 경우) 모두 기술되었는지 확인한다.

[Tip]

- FR 개수가 많으면 줄이기 (4~6 적당)
 - . 필요 시 Appendix에 언급
 - . 신규 기능 위주로
 - . 추상화 레벨은 비슷하게 유지
- FR별 설명을 위한 간단한 도식도
(심사관 이해도 ↑)

[Do]

- 제약 사항은 **설계에 영향을 주는 것을 선정**
- 과제 목적에 맞게 도출 "**일관성**"

2. 요구사항

• 요구사항 도출 (2/2)

- (Use Case도출이 과제 성격에 적절한 경우) 기능 요구 사항들로부터 Use Case가 모두 도출되고 Use Case Diagram이 작성되었는지 확인한다.
 - 기능 요구사항이 모두 설명 가능 해야 한다.
- 요구 사항과 Use Case Diagram으로부터 Domain Model이 작성되었는지 확인한다.
 - 아키텍처 설계 시 Bias되지 않도록 기능위주로 작성되어야 한다.
 - 시스템 경계가 구분되어 있어야 한다.

[Tip]

- 요구사항 도출, 시스템 설명의 목적으로 Use Case Diagram Domain Model등 Diagram 적극 활용
(너무 Detail하여 흐름에 방해가 된다면 Appendix로)

2. 요구사항

- **품질속성 선정**

- 품질 속성과 관련된 시나리오가 전부 작성되었는지 확인한다.
 - 품질 속성별 6 parts scenario(Source, Stimulus, Artifact, Environment, Response, Response Measure)이 모두 기술되어야 한다.
- 품질 속성별 특성에 맞게 측정 가능한 Metric이 도출되고, 측정 기준이 타당한지 확인한다.
 - Metric은 품질 속성별 설계 유효성 검증 시 사용되어야 한다.
- Utility Tree를 활용해 품질 속성 우선 순위가 결정되었는지 확인한다.
 - Utility Tree가 작성되고(Utility – QA – Refined QA – Scenario, Priority)
 - Importance와 Difficulty/Risk 수준이 합리적으로 결정되어야 한다.

2. 요구사항

• 품질속성 선정

- 품질 속성과 관련된 시나리오가 전부 작성되었는지 확인
 - 품질 속성별 6 parts scenario(Source, Stimulus, Artifact, Environment, Priority, Condition)
- 품질 속성별 특성에 맞게 측정 가능한 Metric이 도출되도록 선정
 - Metric은 품질 속성별 설계 유효성 검증 시 사용되어야 한다.

[Don't]

- 개발자 입장의 우선 순위 도출
- **느슨한 기준**으로 판단될 수 있는 QA
(수치가 도전적일 필요는 없지만,
너무 보수적이면 조작의 의심을 받을 수도 있음)
- 품질속성을 충분히 식별하지 않음
(발표를 위해 줄였다면 Appendix로 근거 남기기)
- 우선순위가 중요도/난이도로부터 도출되지 않음
- 동일한 우선 순위의 품질속성을 선택하지 않음
- 하나의 QA에 여러 개의 Metric

[Do]

- 과제 목적과 부합, 설계와 검증까지 **일관성**
- QA 도출 당위성, 논리, 기준 중요
- **Metric 선정** 매우 중요
 - 측정 가능(정량적), 선정 근거 명확해야 함
- QA 우선 순위가 합리적이어야 함

[Tip]

- 선정된 QA가 너무 많지 않도록 (3~6개)
필요 시 근거는 Appendix

2. 요구사항

• 아키텍처 드라이버 선정

- 아키텍처 드라이버(Architectural Drivers)는 아키텍처 설계와 연관된 기능/품질/제약 요구사항들로 선정되었는지 확인한다.
 - Architecture Impact와 Business/Mission Value를 고려해서 Architecturally Significant Requirements(ASRs)로 선정해야 한다..
 - 아키텍처 드라이버들은 설계 시에 반드시 고려되어야 한다.

[Don't]

- 과제에 적절하지 않은 FR, QA 선정
(평가자 예상에 부합하지 않는)
- 설계에 반영되지 않은 요구사항 포함

3. 설계

• 아키텍처 분석

- System Context는 시스템을 제대로 반영하여 작성되었는지 확인한다.
 - 시스템 경계, 시스템과 연동되는 모든 외부 시스템 요소, 외부 시스템과의 동작 등이 정의되어 있어야 함.
- Skeleton Architecture를 정의하고 이를 설명할 수 있는 View들을 기술했는지 확인한다.
 - Architecture를 설명할 수 있는 View를 선택했는지 확인한다 (예, Functional, Information, Behavior, Deployment Views)
 - 각 View가 용도에 맞게 해당 시스템의 내용을 제대로 반영하고 있는 지 확인한다.
 - 각 View를 통해서 해당 시스템을 이해할 수 있어야 한다.
- 분석 결과를 바탕으로 Architectural Drivers를 만족하기 위한 설계 포인트가 선정되었는지 확인한다.

[Do]

- SW구조와 System boundary를 대략적으로 설명
 - . 과제 **범위**, 관심, 구조, 동작을 쉽게 설명 목적
 - . 추후 DP설명으로 이어질 수 있도록 구성
- 많은 고민/노력이 들어간 설계활동들이 드러나도록 (후보안, 각종 View, 최소 Appendix에 포함)

3. 설계

- 아키텍처 설계 안 도출(1/2)

- 설계 포인트 별로 후보 설계 안들이 도출되었는지 확인한다.
 - 후보 설계 안은 Architectural Drivers를 만족하여야 한다.
 - 후보 설계 안의 설계 전략(Tactics/ Patterns 등)이 논리적으로 선정되었는지 확인한다.
 - 도출된 후보 설계 안들은 의미가 있어야 한다 (단순 조합, 개수 늘리기는 지양)
- 후보 설계 안 별로 검토되어야 할 품질 속성이 누락되지 않았는지 확인한다.
- 후보 설계 안 별 선정된 품질 속성들에 대한 영향도가 합리적으로 검토되었는지 확인한다.
 - 영향도 검토 시, 검토결과는 합리적인 근거에 기반해야 함.
 - 필요 시, 근거를 명시적으로 기술해야 함.

3. 설계

- 아키텍처 설계 안 도출(2/2)

- 후보 설계 안들을 다양한 관점(Architectural Drivers)에서 상호 비교 분석하고 비교 기준이 합리적이며, 근거가 명확한지 확인한다 (Pros/Cons 표)
- 최종 설계 안이 합리적으로 선정되었는지 확인한다.
 - 단순히 장점이 많거나, 단점이 적다는 이유로 선정하는 것은 지양한다.
- 선정된 설계 안들이 최종 구조로 통합되었는지 확인한다.

[Do]

- 최종 설계 구조에 선정 설계안들이 적용
 - . 다양한 View, 최종 구조 설계 사례 (Legend 충실)
 - . 최종 Architecture, 설계 반영 전/후 비교

- **구현**

- 주요 컴포넌트 별 누락된 인터페이스가 없는지 확인한다.
- 인터페이스에 오류가 없는 지 확인한다.
상세설계 및 구현 시 필요한 고려사항이 명확히 기술되었는지 확인한다.

- **시스템 검증**

- 모든 요구 기능들이 정상 동작하는지 검증 시나리오(TC) 기반 시험 결과로 판단한다.

- **품질속성 검증**

- 아키텍처 드라이버로 선정된 품질속성이 품질 시나리오에 기술된 Metric 기준으로 모두 만족하는지 확인한다.
 - 만약, 만족하지 않는 품질속성이 있다면, 타당한 근거와 향후 대안이 기술되어야 한다.
타당한 근거가 없다면, 과제 목표를 달성하지 못한 것이다.

4. 구현 / 5. 검증

• 구현

- 주요 컴포넌트 별 누락된 인터페이스가
- 인터페이스에 오류가 없는 지 확인한다.
상세설계 및 구현 시 필요한 고려사항이

[Do]

- 주요 기능요구사항, 품질 속성에 대한 검증 결과
 - . 필요 시 Metric 측정 과정 설명
 - . 평가 결과에 대한 근거 제시

• 시스템 검증

[Tip]

- 기술 리더십
 - . 개발자의 성장 독려, 코드 품질 향상, 커뮤니케이션 등
 - . 수행한 과제에서 나의 Architect 역할 정리
 - . 향후 계획
- 성과
 - . 과제 배경 / 필요성 / 목표에서 기술한 내용 관점에서 과제 목적이 달성되었음을 어필
 - . 데모: 실제 제품에 들어갔다는 것도 매우 중요한 성과
 - . 특허, 시상, 비즈니스적 성과 정리

지 확인

- **리허설 / 멘토링 / 리뷰**

- 면접 심사 형식으로 리허설 / 리뷰 반복하기 (최소 2회 이상)
- 여러 리뷰어의 유사한 피드백은 최대한 수용
- 다양한 리뷰어로부터 리뷰 받기 (임원 / 동료 / 팀과제원 / 과제 무관한 사람/ 다른 사업부 Architect / 가족)

- **발표**

- 정확한 Terminology 사용
- 이야기의 흐름 준비하여 내용의 일관성 유지
과제의 중요성, 요구사항, 설계 문제, Trade off, 결정과 결과를 쉽고 명확하게 전달
- 예상 질문에 대한 답변을 위한 Appendix 준비 (Hyperlink)
- 긍정적, 능동, 간결한 언어 사용
cf. 부정적, 수동/3인칭, 습관적 불필요 추임새, 불필요한 언급(컨디션 난조, 업무 바쁨)

- 발표 자료

- 제목에서 과제가 설명될 수 있게, 과제를 이해할 수 있는 키워드 포함
(↔ 잘못된 선입견, 오해를 야기할 수 있는 단어를 제목에 사용)
- 기본 설계 활동 충실히 수행했음 Appeal
(Use Case, System Context, Domain Model 제공, 최종 설계, Sequence Diagram, Class Diagram 등)
- 불명확한 표현 사용하지 말 것 (ex: 다른 기능 확장 → 다른 기능은 무엇인가?)

- 멘탈 관리

- 모두 Architect로서의 역량을 가지고 있습니다(AA, A입과)
- 포기 금지 !!! (이 또한 지나 갑니다)
- 모의 면접 → 본인 성향에 맞춰 (많이 하는게 좋을 수도 아닐 수도. 멘탈 관리가 더 중요할 수도)

• 과제 상관 없이 자주 등장하는 질문

- 과제 진행하면서 가장 어려웠던 점? 어떻게 해결했는가?
- DP 후보안 들에 대한 상세 질문, 무엇을 고민했고 어떤 근거로 선택했는지?
가장 어려웠던 점과 극복 방안 (낮은 Metric 개선) 계획
- Stakeholder 요구사항 수집 Communication을 어떻게 하였는지?
- 기존에 없던 새로운 방식을 적용한 사례가 있는가? 실제로 본인이 진행한 일은 무엇인가?
- 본인이 생각하는 Architect의 역할은 무엇인가? Architect로써 가장 잘 한 일은 무엇인가?

• 질의 응답

- 질문의 의도에 맞는 답하기
 - 정확한 설계 용어 사용하여 두괄식 답변
 - 질문의 의도가 명확하게 이해되지 않는다면 이해한 언어로 표현하여 질문의 의도가 맞는 지 확인 후 답변
(질문이 길다면 질문의 요점을 정리 후 질문 내용이 맞는 지 확인 후 답변)
- 동문서답 피하기
 - 방어적 답변 지양: 핑계, 3인칭
 - 불필요한 답변: 설계 단점 굳이 자진 토로

Thank You